Worksheet No. 5

Student Name: ANINDITA DHAR **UID:** 25MCA20259
Branch: MCA (GENERAL) **Section/Group:** MCA- 1-A
Semester: II **Date of Performance:** 23/01/2026

Subject Name: Technical Training **Subject Code:** 25CAP-652

1.Aim/Overview of the practical:

To gain hands-on experience in creating and using cursors in PostgreSQL for row-by-row data processing, enabling sequential access and manipulation of query results.

2. Objectives:

- To understand the concept and need of cursors in PostgreSQL
- To learn sequential data access using cursors
- To perform row-level manipulation using cursor logic
- To understand cursor lifecycle management
- To handle cursor termination and exceptions.

3. Input/Apparatus Used:

- PostgreSQL
- pgAdmin

4. Practical / Experiment Steps

Pre-requisite Understanding

Step 1: Implementing a Simple Forward-Only Cursor
Creating a cursor to loop through an Employee table and print individual records.
Step 2: Complex Row-by-Row Manipulation
Using a cursor to update salaries based on a dynamic "Experience-to-Performance" ratio logic.
Step 3: Exception and Status Handling
Ensuring the cursor handles empty result sets or termination signals gracefully

4.Procedure/Algorithm/Code:

i.Start the system and log in to the computer.
ii. Open PostgreSQL software.

iii. **Create a table using the DDL command.**

```
CREATE TABLE Employees_1(
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    salary INT,
    experience INT,
    performance VARCHAR(1)
);
```

iv. Insert records into the table .

```
INSERT INTO Employees_1 VALUES
(1, 'Aninidta', 25000, 5, 'B');

INSERT INTO Employees_1 VALUES
(2, 'Kunal', 40000, 3, 'A');

INSERT INTO Employees_1 VALUES
(3, 'Atharva', 25000, 2, 'C');

INSERT INTO Employees_1 VALUES
(4, 'Vandana', 30000, 4, 'A');

INSERT INTO Employees_1 VALUES
(5, 'Sakhi', 30000, 3, 'B');
```

vi.

```
DO $$
DECLARE
   emp_cursor CURSOR FOR
      SELECT emp_id, emp_name, salary FROM Employees_1;
   rec RECORD;
BEGIN
   OPEN emp_cursor;
   LOOP
      FETCH emp_cursor INTO rec;

      EXIT WHEN NOT FOUND;

      RAISE NOTICE 'ID: %, Name: %, Salary: %',
      rec.emp_id, rec.emp_name, rec.salary;
   END LOOP;
```

```
    CLOSE emp_cursor;
END $$;
```

```
NOTICE:  ID: 1, Name: Aninidta, Salary: 25000
NOTICE:  ID: 2, Name: Kunal, Salary: 40000
NOTICE:  ID: 3, Name: Atharva, Salary: 25000
NOTICE:  ID: 4, Name: Vandana, Salary: 30000
NOTICE:  ID: 5, Name: Sakhi, Salary: 30000
DO

Query returned successfully in 147 msec.
```

viii.

```
DO $$
DECLARE
    emp_cursor CURSOR FOR
        SELECT emp_id, salary, experience, performance FROM Employees_1;
    rec RECORD;
    new_salary NUMERIC;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO rec;
        EXIT WHEN NOT FOUND;

        IF rec.experience >= 5 AND rec.performance = 'A' THEN
            new_salary := rec.salary * 1.20;
        ELSIF rec.experience >= 3 AND rec.performance = 'B' THEN
            new_salary := rec.salary * 1.10;
        ELSE
            new_salary := rec.salary * 1.05;
        END IF;

        UPDATE employee
        SET salary = new_salary
        WHERE emp_id = rec.emp_id;
    END LOOP;
    CLOSE emp_cursor;

END $$;
```

```
DO

Query returned successfully in 74 msec.
```

ix.

```
DO $$
DECLARE
   emp_cursor CURSOR FOR SELECT * FROM Employees_1;
   rec RECORD;
BEGIN
   OPEN emp_cursor;
   LOOP
      FETCH emp_cursor INTO rec;
      EXIT WHEN NOT FOUND;
      RAISE NOTICE 'Processing Employee: %', rec.emp_name;
   END LOOP;
   CLOSE emp_cursor;

EXCEPTION
   WHEN OTHERS THEN
      RAISE NOTICE 'Error occurred: %', SQLERRM;
END $$;
```

```
DO

Query returned successfully in 74 msec.
```

7. Learning Outcomes

- **Cursor Implementation:** Students will be able to design, implement, and manage cursors to solve row-wise processing problems.
- **Lifecycle Mastery:** Students will demonstrate the correct syntax for declaring, opening, fetching, and closing cursors.
- **Error Prevention:** Students will understand how to properly handle row-by-

row processing exceptions and prevent memory leaks via deallocation.

- **Analytical Thinking:** Students will be able to apply cursor-based logic to solve real-world scenarios like multi-level payroll adjustments or data migrations.